

Предефиниране на оператори

Лекция 7

Операторите са унарни (с един операнд) и бинарни (с два операнда). Характеризират се с позицията на оператора спрямо операнда/и, приоритет и асоциативност.

Не можем да създаваме нови оператори, можем само да предефинираме съществуващи.

Оператори в C++

- Позицията на оператора спрямо операнда/ите му го определя като:
 - префиксен ($\oplus a$)
 - инфиксен ($a \oplus b$), $(a * b)$
 - постфиксен ($a \oplus$)
- Приоритетът определя реда на изпълнение на операторите в израза.

Предефиниране на оператори - позволява стандартните C++ оператори да работят с потребителски типове. Така обектите се държат като вградени типове

Предефинираме оператор, когато:

- операторът има естествен смисъл
- семантиката е ясна.

Не трябва да се променя смисъла на оператора

Не можем да предефинираме:

- - оператор за избор на член от клас
- * - оператор за избор на член на клас чрез указател
- :: - оператор за присъединяване
- ? - троен условен израз
- sizeof

Ключова дума friend - за семантични оператори ($==$, $<<$)
- когато не искаме getter-и
- при тясно свързани помощни класове

Defaulted operator ($<=>$) - Вместо да дефинираме всички в оператора за сравнение ($==$, $!=$, $<$, $>$, $<=$, $>=$)

• Връща `auto`, защото резултатът е един от следните типове:
`std::strong_ordering`, `std::weak_ordering`, `std::partial_ordering`

Auto functions in C++

↳ ключова дума за автоматично извеждане на типове (типът се извежда от компилатора)

Задължително трябва да има инициализация, инак типът няма да бъде изведен.

& и `const` се губят, ако изрично не се напишат.

Използваме при:

- дълги и сложни типове
- `template`
- итератори
- `lambda` функции